

■ 1. The function of OS is \_\_\_\_\_.

A. explain and execute source program

B. compile source program

C. switch coding

D. control and manage system source

■ 2. The goal of a time-sharing operating system pursues is\_\_\_\_\_.

A. Response immediately

B. Multi-thread

C. Velocity

D. Parallel execute

■ 3. Which one is not the issue of handheld system?

- A. Limited memory
- B. Slow processor
- C. Small display screens
- D. Not a real-time system

■ 4. Which operating system allows interaction between user and process?

A. Batch mode OS

B. Multiprogrammed batch OS

C. Time-shared OS

D. Real-time OS

■ 5. Which of the following application should not be classified as batch-oriented?

A. Word processing

B. Generating monthly bank statements

C. Generate employee w-2 tax forms

D. Computing pi to million decimal places

■ 6. From the computer's point of view, the operating system is the program most intimately involved with the hardware. In this context, we can view an operating system as a \_\_\_\_\_.

A. resource allocator

B. workstation

C. server

D. interface

# Chapter 2: Operating-System Structures

- Operating System Services
- User Operating System Interface
- System Calls(系统调用)
- Types of System Calls
- System Programs
- Operating System Design and Implementation
- Operating System Structure
- Virtual Machines
- Operating System Generation
- System Boot (系统引导)

# Operating System Services

- User interface - Almost all operating systems have a user interface (UI)  
Command-Line (CLI), Graphics User Interface (GUI)
- Program execution – system capability to load a program into memory and to run it.
- I/O operations – since user programs cannot execute I/O operations directly, the operating system must provide some means to perform I/O.
- File-system manipulation – program capability to read, write, create, and delete files.



- Communications – exchange of information between processes executing either on the same computer or on different systems tied together by a network. Implemented via *shared memory*(共享内存) or *message passing*(消息传递).
- Error detection – ensure correct computing by detecting errors in the CPU and memory hardware, in I/O devices, or in user programs.

# Additional Operating System Functions

Additional functions exist not for helping the user, but rather for ensuring efficient system operations.

- Resource allocation(资源分配) – allocating resources to multiple users or multiple jobs running at the same time.
- Accounting(记账) – keep track of and record which users use how much and what kinds of computer resources for account billing or for accumulating usage statistics.
- Protection – ensuring that all access to system resources is controlled.

# User Operating System Interface - CLI

CLI allows direct command entry

- Sometimes implemented in kernel, sometimes by systems program
- Sometimes multiple flavors implemented
  - **shells**

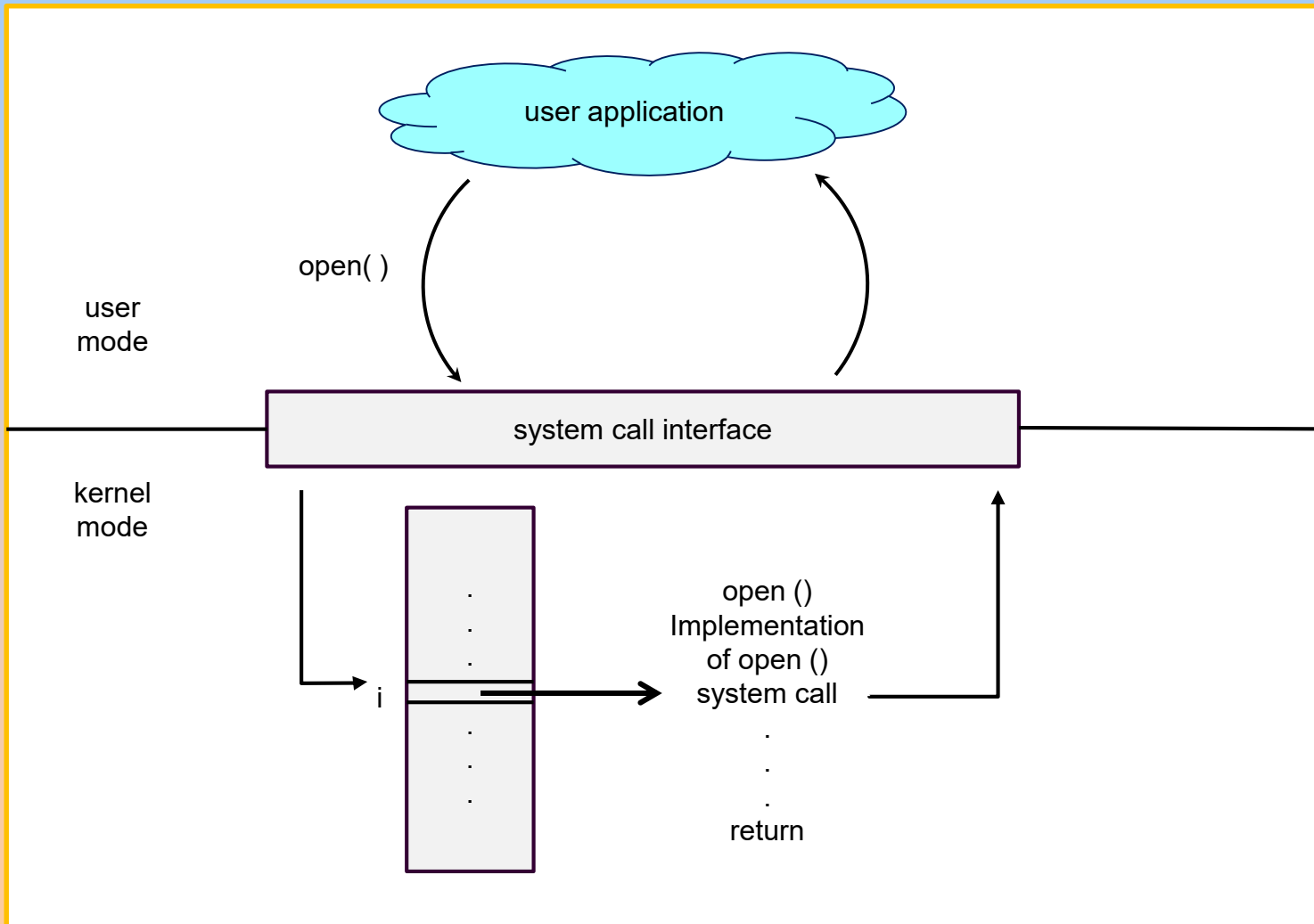
# User Operating System Interface - GUI

- User-friendly **desktop** metaphor interface
  - ☞ Usually mouse, keyboard, and monitor
  - ☞ Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
  - ☞ Microsoft Windows is GUI with CLI “command” shell
  - ☞ Solaris is CLI with optional GUI interfaces

# System Calls(系统调用)

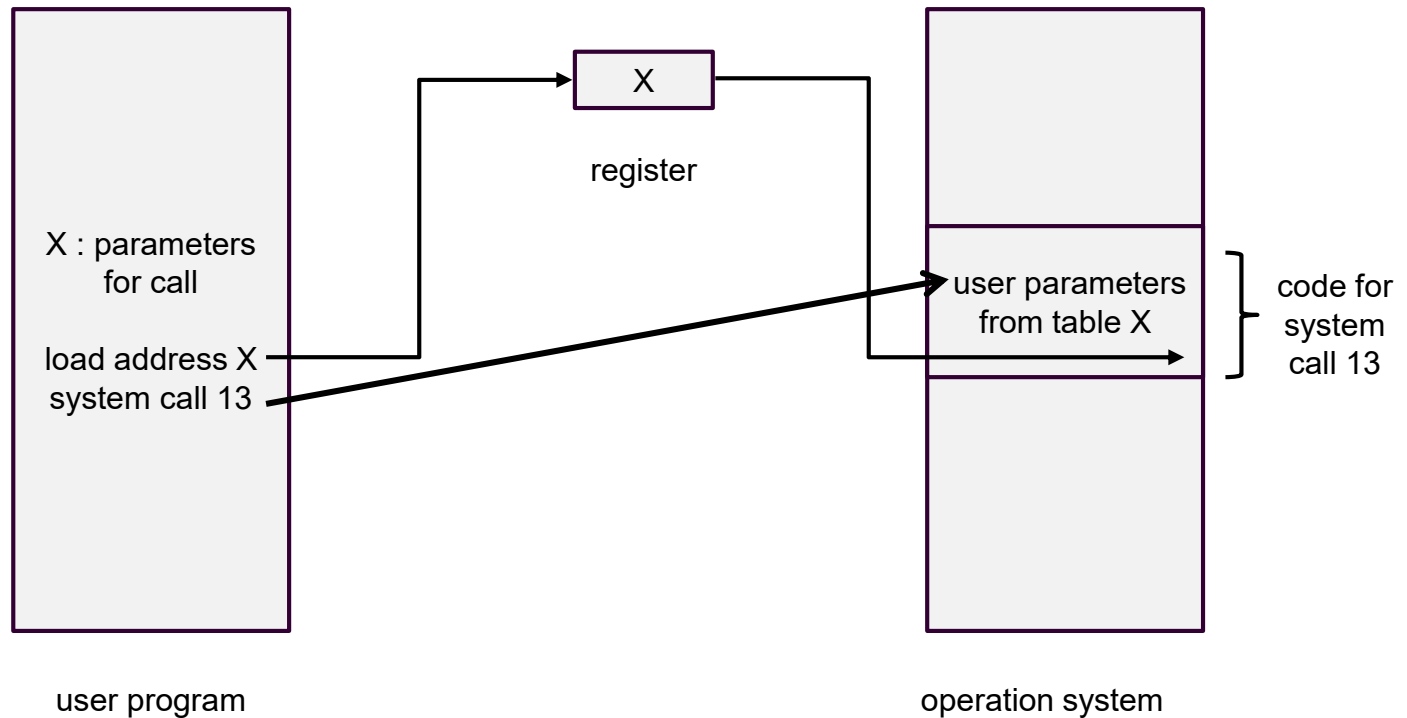
- System calls provide the interface between a running program and the operating system.
  - ☞ Generally available as assembly-language instructions.
  - ☞ Languages defined to replace assembly language for systems programming allow system calls to be made directly (e.g., C, C++)

# API – System Call – OS Relationship



- Three general methods are used to pass parameters between a running program and the operating system.
  - ☞ Pass parameters in *registers*(寄存器).
  - ☞ Store the parameters in a table in memory, and the table address is passed as a parameter in a register.
  - ☞ *Push*(压入) (store) the parameters onto the *stack* by the program, and *pop* off the stack by operating system.

# Passing of Parameters As A Table

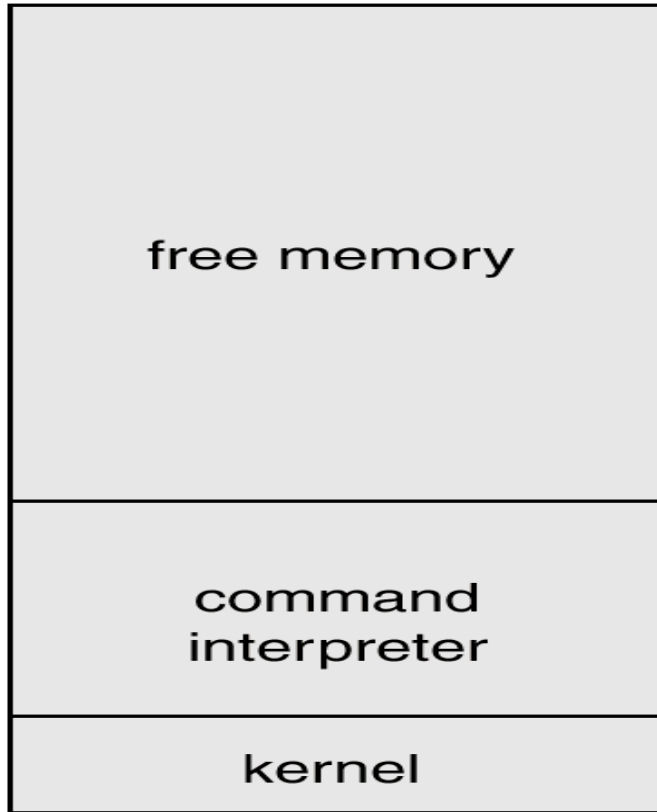




# Types of System Calls

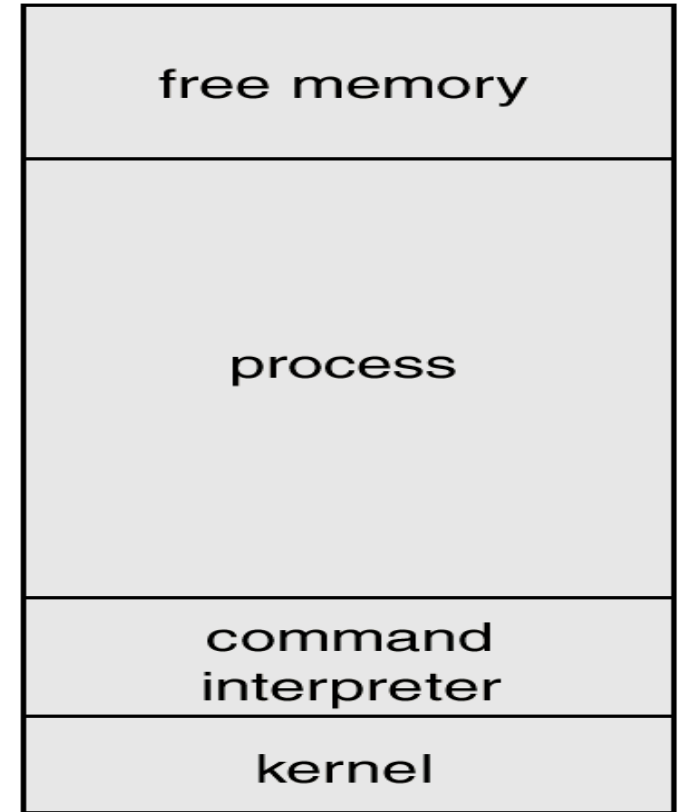
- Process control(进程控制)
- File management
- Device management
- Information maintenance
- Communications

# MS-DOS Execution



(a)

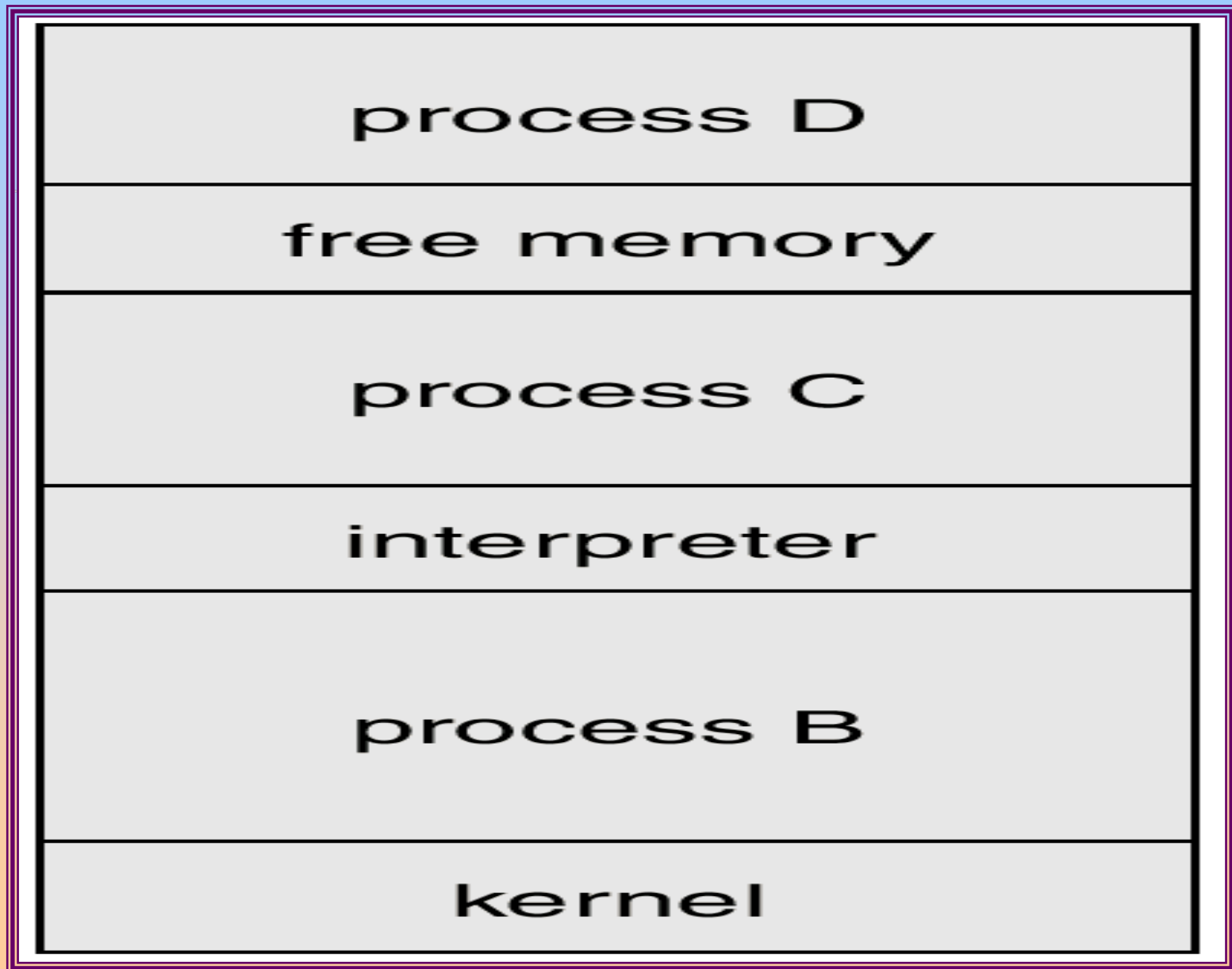
At System Start-up



(b)

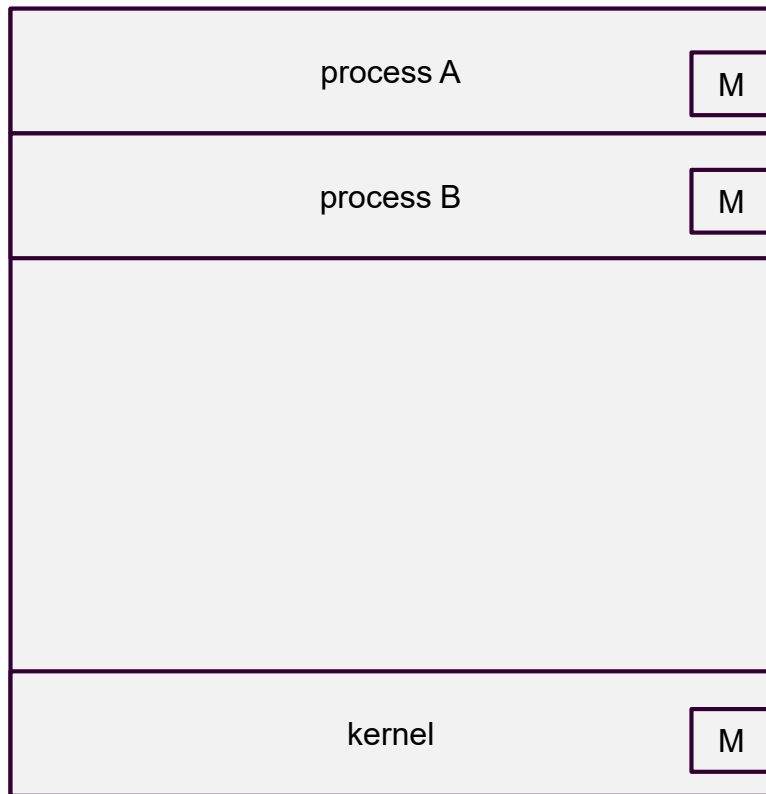
Running a Program

# UNIX Running Multiple Programs

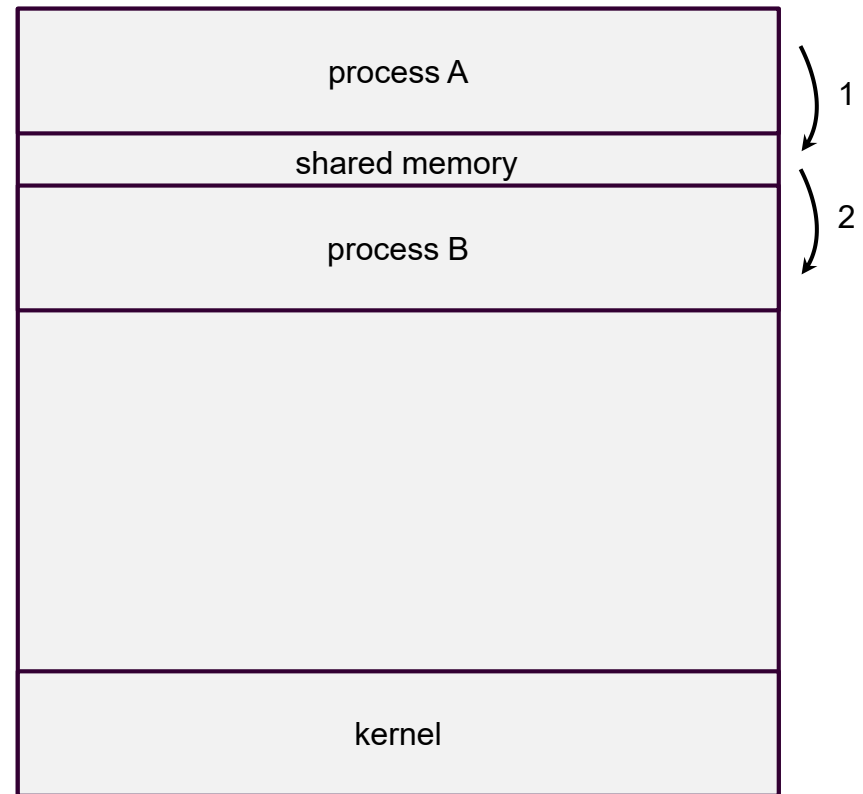


# Communication Models

- Communication may take place using either message passing or shared memory.



Msg Passing



Shared Memory

# System Programs

- System programs provide a convenient environment for program development and execution. They can be divided into:
  - ➡ File manipulation
  - ➡ Status information
  - ➡ File modification
  - ➡ Programming language support
  - ➡ Program loading and execution
  - ➡ Communications
  - ➡ Application programs
- Most users' view of the operation system is defined by system programs, not the actual system calls.

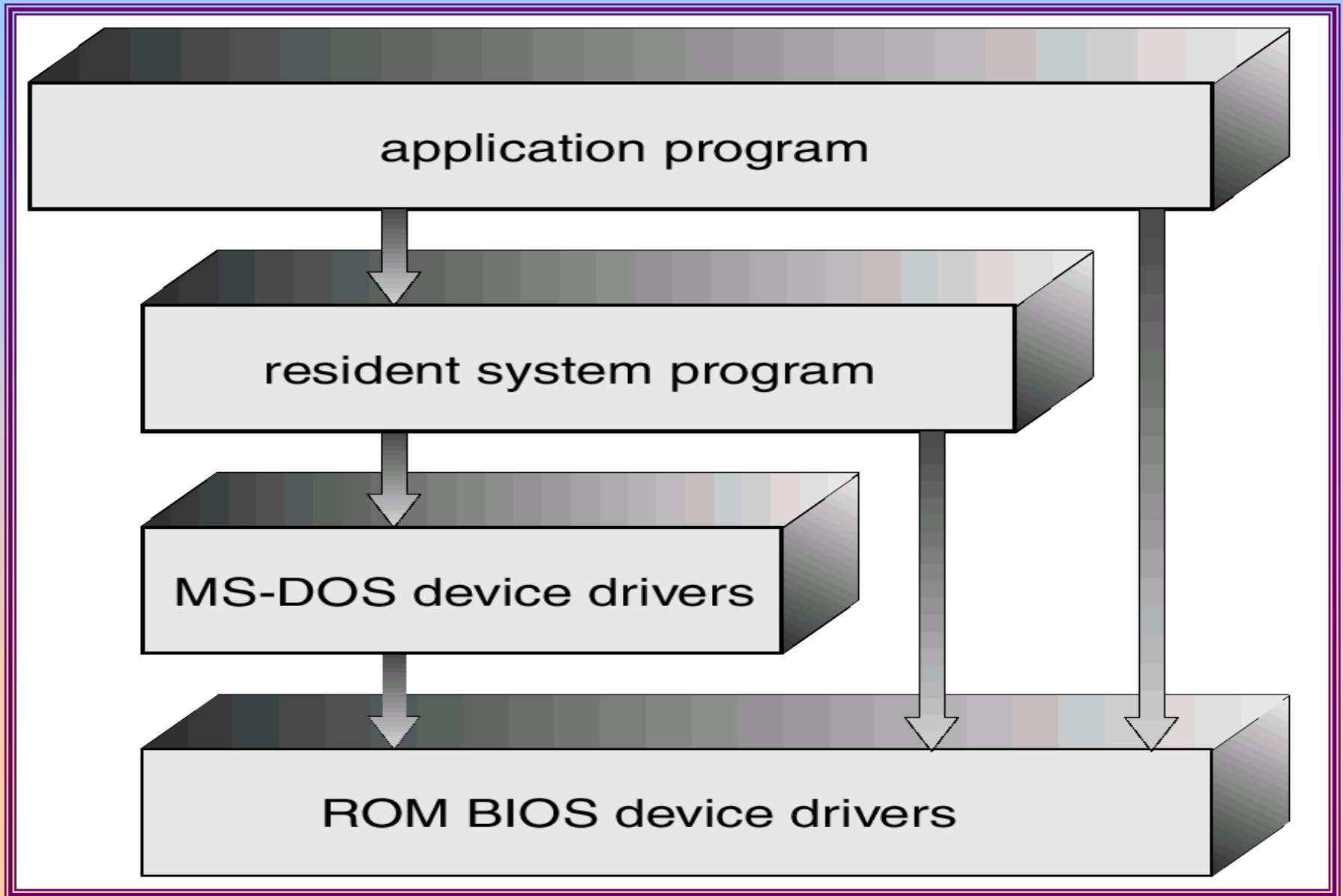
# Operating System Design and Implementation

- Start by defining goals and specifications
- Affected by choice of hardware, type of system
- *User goals and System goals*
  - ☞ User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
  - ☞ System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

# MS-DOS System Structure

- MS-DOS – written to provide the most functionality in the least space
  - ☞ not divided into modules
  - ☞ Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated

# MS-DOS Layer Structure





# UNIX System Structure

- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts.

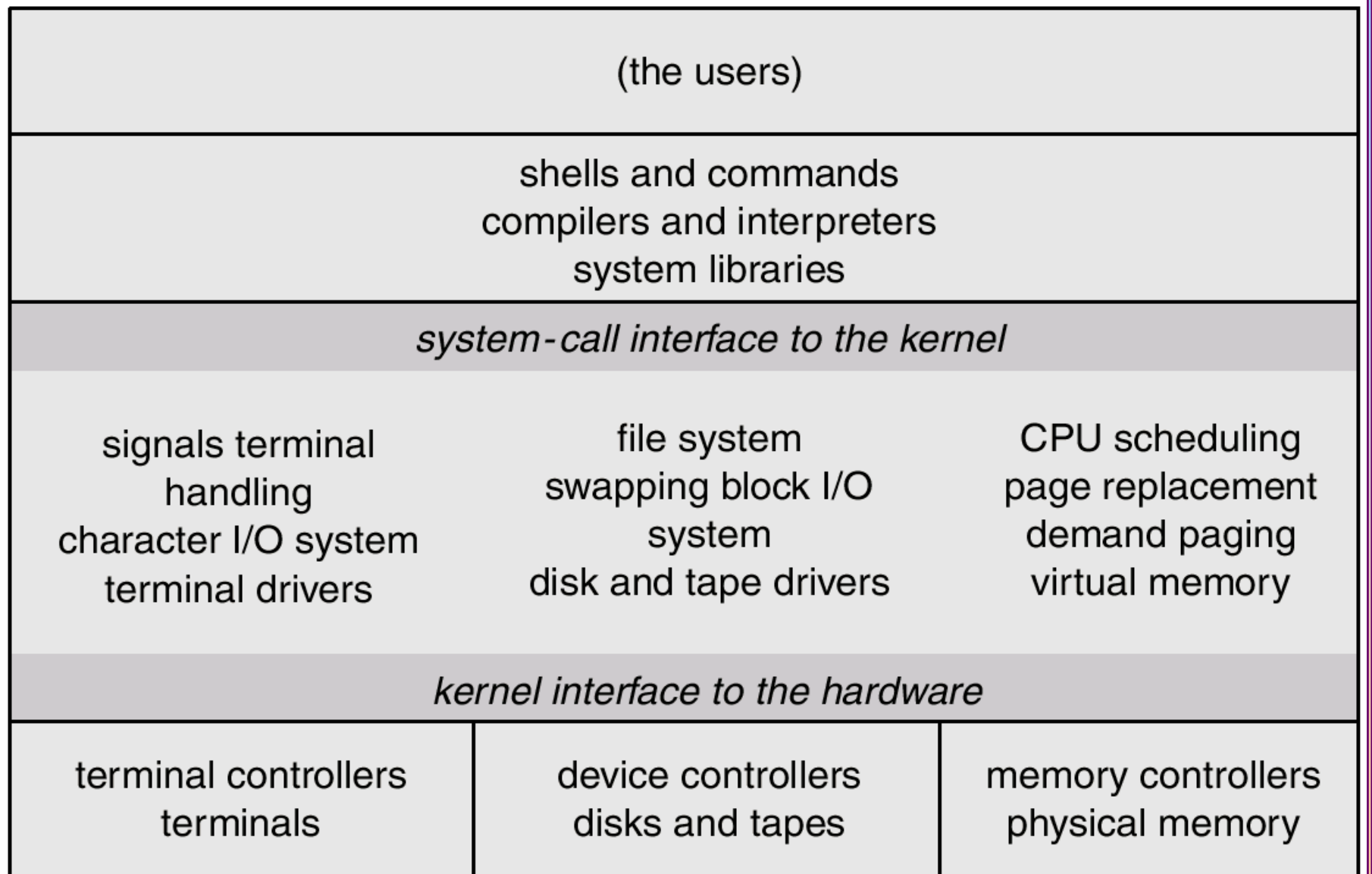
- ☞ Systems programs

- ☞ The kernel

- ☞ Consists of everything below the system-call interface and above the physical hardware

- ☞ Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level.

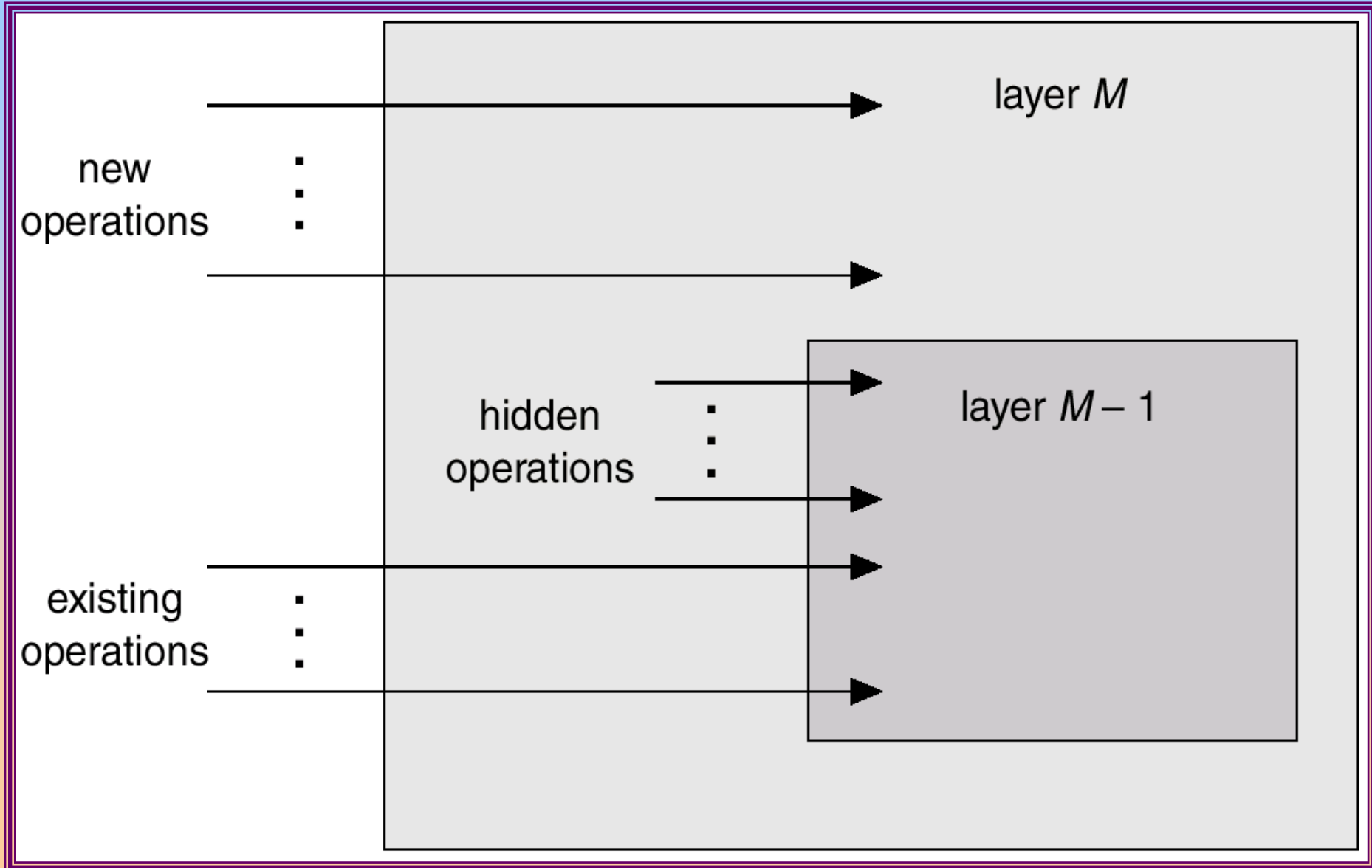
# UNIX System Structure



# Layered Approach(分层法)

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers.

# An Operating System Layer



# Microkernel System Structure

- Moves as much from the kernel into “*user*” space.
- Communication takes place between user modules using message passing.
- Benefits:
  - easier to extend a microkernel (微内核)
  - easier to port the operating system to new architectures
  - more reliable (less code is running in kernel mode)
  - more secure

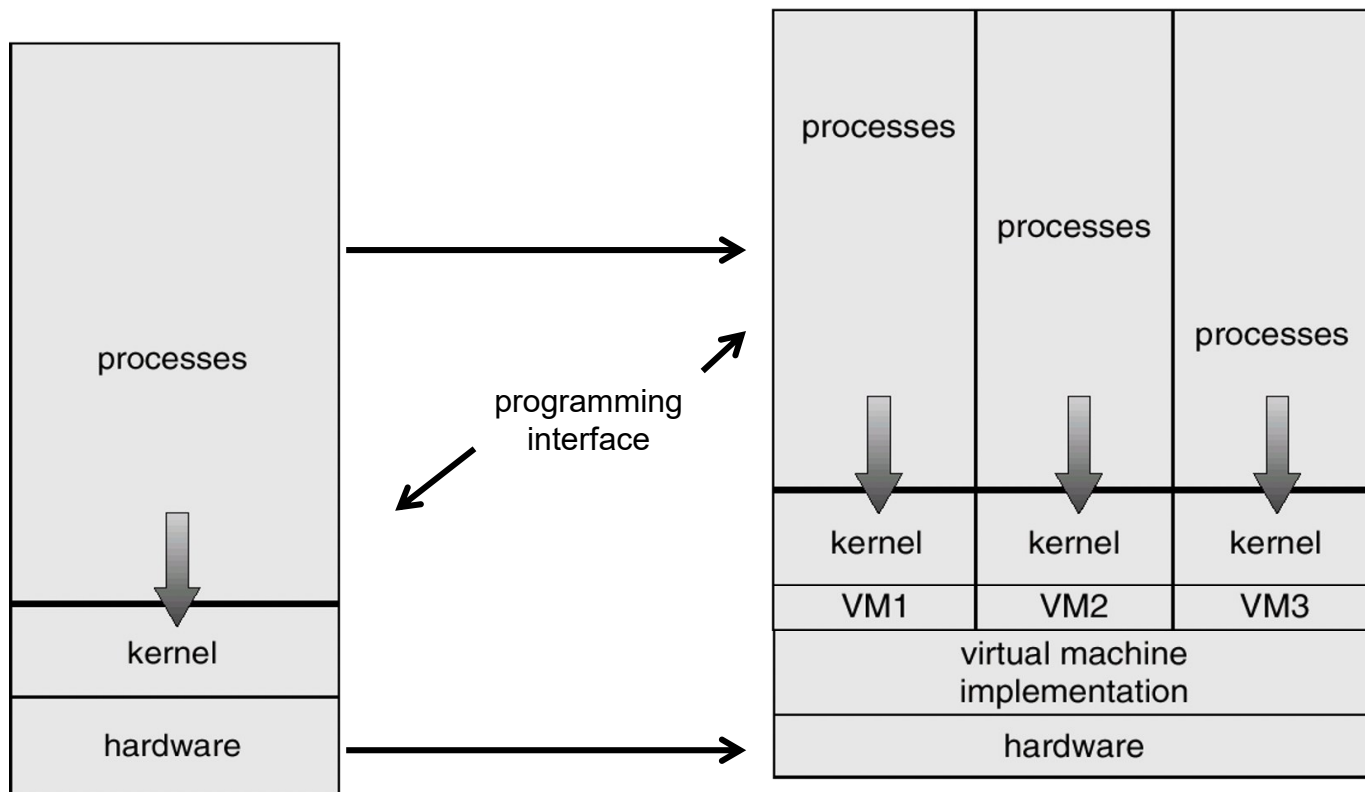
# Virtual Machines(虚拟机)

- A *virtual machine* takes the layered approach to its logical conclusion. It treats hardware and the operating system kernel as though they were all hardware.
- A virtual machine provides an interface *identical* to the underlying bare hardware.
- The operating system creates the illusion of multiple processes, each executing on its own processor with its own (virtual) memory.

# Virtual Machines (Cont.)

- The resources of the physical computer are shared to create the virtual machines.
  - ☞ CPU scheduling can create the appearance that users have their own processor.
  - ☞ Spooling and a file system can provide virtual card readers and virtual line printers.
  - ☞ A normal user time-sharing terminal serves as the virtual machine operator's console.

# System Models



Non-virtual Machine

Virtual Machine



# Advantages/Disadvantages of Virtual Machines

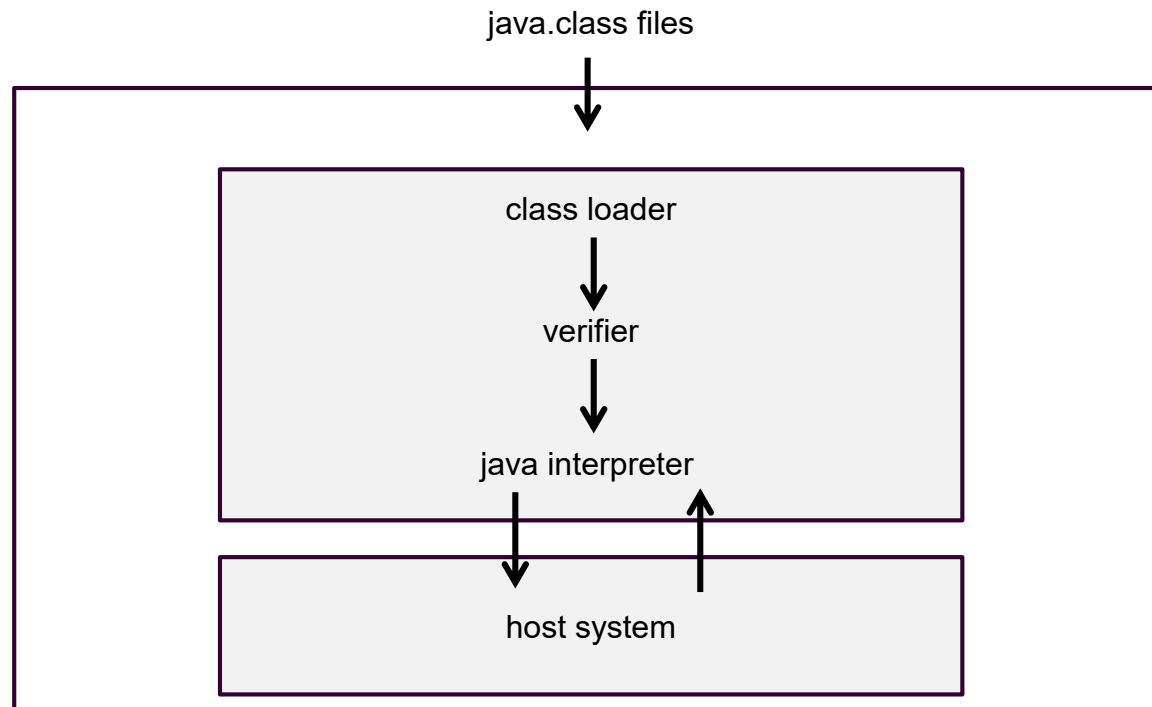
- The virtual-machine concept provides complete protection of system resources since each virtual machine is isolated from all other virtual machines. This isolation, however, permits no direct sharing of resources.

- A virtual-machine system is a perfect vehicle for operating-systems research and development. System development is done on the virtual machine, instead of on a physical machine and so does not disrupt normal system operation.
- The virtual machine concept is difficult to implement due to the effort required to provide an *exact* duplicate to the underlying machine.

# Java Virtual Machine

- Compiled Java programs are platform-neutral bytecodes executed by a Java Virtual Machine (JVM).
- JVM consists of
  - class loader
  - class verifier
  - runtime interpreter
- Just-In-Time (JIT) compilers increase performance

# Java Virtual Machine



# System Design Goals

- User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast.
- System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient.

# Mechanisms and Policies

- Mechanisms determine how to do something, policies decide what will be done.
- The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later.

# System Implementation

- Traditionally written in assembly language, operating systems can now be written in higher-level languages.
- Code written in a high-level language:
  - ☞ can be written faster.
  - ☞ is more compact.
  - ☞ is easier to understand and debug.
- An operating system is far easier to *port*(端口) (move to some other hardware) if it is written in a high-level language.

# System Generation (SYSGEN)

- Operating systems are designed to run on any of a class of machines; the system must be configured for each specific computer site.
- SYSGEN program obtains information concerning the specific configuration of the hardware system.
- *Booting* – starting a computer by loading the kernel.
- *Bootstrap program* – code stored in ROM that is able to locate the kernel, load it into memory, and start its execution.



初始引导过程主要由计算机的BIOS完成。BIOS是固化在ROM中的基本输入输出系统（Basic Input/Output System），其内容存储在主板ROM芯片中，主要功能是为内核运作环境进行预先检测。

其功能主要包括中断服务程序、系统设置程序、上电自检和系统启动自举程序等

**BIOS（Basic Input—Output System）**基本输入输出系统，其内容集成在微机主板上的一个ROM芯片上，主要保存着有关微机系统最重要的基本输入输出程序，系统信息设置、开机上电自检程序和系统启动自举程序等。

微机部件配置记录是放在一块可读写的 **CMOS RAM** 芯片中的，主要保存着系统基本情况、**CPU**特性、软硬盘驱动器、显示器、键盘等部件的信息。

**CMOS RAM**芯片由系统通过一块后备电池供电，无论是在关机状态中，还是遇到系统掉电情况，**CMOS**信息都不会丢失。

在 **BIOS ROM**芯片中装有"系统设置程序"，主要用来设置**CMOS RAM**中的各项参数。

■ 2.3

■ 2.7